Qwen

Верификация движка **X-Ray:** От **C++** к **Python** — как воспроизвести идентичную логику генерации конфигураций без запуска игры

Архитектурный контекст и происхождение движка **X-Ray Monolith**

Для проведения точной верификации логики генерации конфигураций необходимо прежде всего деконструировать сам объект анализа — движок X-Ray Monolith. Исследование не может ограничиваться простым сравнением двух кодовых баз; оно должно начинаться с понимания того, какой именно движок подлежит анализу. Репозиторий `xrLil-Batya/xray-monolith`, который является основой для данного исследования, представляет собой не "чистый" или первоначальный исходный код, а сложный гибрид, являющийся результатом слияния нескольких проектов и значительных собственных доработок [2] [5]. Это имеет фундаментальное значение, поскольку задача состоит не в воспроизведении некоего идеального "оригинала", а в точном имитировании логики именно этой конкретной, модифицированной версии движка, которая используется в популярном моде S.T.A.L.K.E.R. Anomaly [10].

Происхождение `xray-monolith` можно проследить через цепочку зависимостей и форков. В основе лежит OpenXRay Call of Chernobyl Edition, который сам по себе является продолжением развития X-Ray Engine, использовавшегося в играх серии S.T.A.L.K.E.R. [3] [5]. Более точно, репозиторий основан на X-Ray 1.6 Engine, версия 1.6.02, которая была основой для S.T.A.L.K.E.R.: Call of Pripyat [2] [5]. Однако движок не остановился на этом. Авторы `xray-monolith` внесли изменения, заимствованные из S.T.A.L.K.E.R.: Call of Pripyat 1.4.22, что еще больше усложняет его генеалогию [5]. Кроме того, в репозитории присутствуют коммиты из IX-Ray Engine, что указывает на использование дополнительных передовых технологий и исправлений от других сообществ разработчиков [1]. Таким образом, `xray-monolith` — это не

просто один исходник, а консолидированное решение, объединившее лучшие черты нескольких проектов, направленное на создание более стабильной и функциональной среды для моддинга.

Ключевой особенностью этого движка, определяющей всю сложность задачи, является внедрение мощных систем моддинга, таких как Dynamic LTX (DLTX) и Dynamic XML (DXML) [1] [2]. DLTX — это система, позволяющая модам безопасно переопределять, дополнять и удалять секции и ключи из базовых `.ltx` файлов, не заменяя их напрямую [2] [3]. Эта система была добавлена в `xray-monolith` благодаря работе MerelyMezz, с последующими правками и исправлениями от демонизированного [2]. DXML, в свою очередь, позволяет изменять XML-файлы с помощью Lua-скриптов перед их обработкой движком [1]. Эти надстройки являются неотъемлемой частью логики, которую необходимо воспроизводить. Любая попытка перенести только базовый парсинг LTX, игнорируя DLTX, неизбежно приведет к получению неверного, неидентичного конфига.

Важно отметить, что авторы репозитория `xrLil-Batya/xray-monolith` явно заявляют об отсутствии официальной связи с разработчиками S.T.A.L.K.E.R. Anomaly, GSC Game World или даже с оригинальным разработчиком мода Sidorovich [5]. Это означает, что движок развивается независимо, но все же находится в тесной связке с конкретной версией мода (например, 1.5.1 или 1.5.2) [4] [5]. Соответственно, для достижения полной идентичности необходимо воспроизводить логику именно этой версии движка. Компиляция этого кода требует использования Visual Studio 2015 или 2022, а также Windows 8.1 SDK, что говорит о его принадлежности к эпохе разработки 2015-2022 годов и потенциально использует специфические для этих версий компиляторов подходы [5] [10].

Таким образом, первоначальный шаг анализа заключается в осознании, что мы имеем дело с "живым" программным обеспечением, которое является продуктом коллективного творчества и эволюции. Задача исследователя — не найти "истинный" источник, а смоделировать его текущее, многослойное поведение. Это требует глубокого погружения не только в исходный код, но и в документацию, форумные обсуждения и историю изменений, чтобы понять, почему тот или иной механизм работает именно так, а не иначе. Например, изменение поведения при попытке переопределить несуществующую секцию с аварийного завершения на запись сообщения в лог — это важный нюанс, который показывает эволюцию движка и должен быть воспроизведен в Python-

реализации для гарантии идентичности [1]. Аналогично, внедрение robin_hood hashing вместо стандартной `xr_unordered_map` указывает на стремление авторов к оптимизации производительности, что, хоть и не влияет напрямую на содержимое конфига, может косвенно влиять на порядок обработки данных [2].

В конечном счете, для успешной верификации необходимо рассматривать `xray-monolith` как единый, целостный продукт. Его архитектура, представленная в виде класса `CInifile` в модуле `xrCore` (`src/xrCore/xr_ini.h` и `src/xrCore/Xr_ini.cpp`), является центральным звеном, который координирует весь процесс [20]. Этот класс не просто парсит файлы, он управляет кэшированием, обрабатывает директивы `#include`, и, что самое главное, реализует всю сложную логику слияния файлов через систему DLTX [20]. Поэтому любой Python-эквивалент должен быть не просто набором функций для чтения `.ltx` файлов, а полноценным симулятором этого класса `CInifile` и всей его экосистемы. Без такого комплексного подхода любые попытки получить идентичный конфиг будут обречены на провал.

Фундаментальные механизмы обработки **LTX** и **XML** конфигураций

Понимание базовых механизмов обработки конфигурационных файлов является отправной точкой для любого анализа, направленного на воспроизведение финального состояния конфигурации. Движок X-Ray Monolith использует два основных текстовых формата для хранения игровых данных: LTX и XML [7]. Хотя оба формата имеют INI-подобную структуру, их жизненные циклы и модели загрузки кардинально различаются, что напрямую влияет на процесс их объединения и, следовательно, на финальный "гигаконфиг".

LTX (`.ltx`) является основным форматом для большинства игровых данных: это настройки предметов, характеристик монстров, параметры торговцев, настроек игры и многое другое [7] [11]. Ключевая особенность обработки LTX-файлов заключается в том, что они загружаются и объединяются в единое хранилище данных один раз во время старта игры [7]. Движок сканирует все директории, указанные в пути поиска, и собирает все `.ltx` файлы. По умолчанию основной каталог для поиска — это `gamedata\config\`, как указано в файле

fsgame.ltx через переменную $game_config$ [12]. После сбора всех файлов движок выполняет их парсинг и слияние в единственный виртуальный файл, известный как system.ltx [7]. Этот процесс является полностью офлайн-процессом, завершающимся перед началом любой игровой сессии или выполнением скриптов. Именно этот объединенный результат и является целью нашего исследования. Поскольку все данные уже собраны в system.ltx при старте, любые последующие операции чтения из конфигурации происходят из этого предварительно сформированного и оптимизированного хранилища [20]. Для нашей задачи, где требуется получить финальный вид конфига без запуска игры, можно сосредоточиться исключительно на LTX-обработке.

XML (.xml), в свою очередь, используется для совершенно иных целей: управления интерфейсом пользователя (UI), хранения строк для локализации, определения диалоговых деревьев и профилей спавна [7]. В отличие от LTX, XML-файлы не предзагружаются и не объединяются в единый файл при старте. Движок читает каждый XML-файл по требованию, то есть только в тот момент, когда соответствующий элемент интерфейса открывается игроком или игровой скрипт обращается к нему [7]. Это принципиальное различие имеет важные практические последствия. Если внести изменения в LTX-файл, эти изменения вступят в силу только после полного перезапуска игры, так как system.ltx был собран один раз и сохранен в памяти [7]. Изменения в XML-файлах могут вступить в силу гораздо раньше — достаточно лишь открыть/закрыть экран, который использует этот файл. Также стоит отметить наличие системы DXML, которая позволяет применять Lua-скрипты для модификации XML-файлов непосредственно перед их чтением, причем эти скрипты могут выполнятся многократно в течение одной игровой сессии [1] [7].

Эти фундаментальные различия в моделях загрузки делают LTX-систему центральным объектом для нашего исследования. Цель получения "гигаконфига" практически эквивалентна задаче симуляции процесса создания файла system.ltx.

Синтаксис самого LTX-файла относительно прост. Он состоит из секций, объявляемых в квадратных скобках, таких как [wpn_ak74]. Внутри каждой секции находятся пары ключ-значение, разделенные знаком равенства (=). Движок поддерживает несколько типов данных: строки, числа с плавающей точкой, целые числа (знаковые и беззнаковые) и логические значения (true/false) [11]. Примечательно, что по умолчанию все значения считаются строками, и для их корректного чтения в нужном типе (например, число)

необходимо использовать специальные функции в Lua, такие как `r_float` или `r_s32` [12]. Также поддерживаются комментарии, начинающиеся с точки с запятой (`;`), и игнорирование пробелов вокруг знака равенства [11].

Одной из самых мощных и сложных особенностей LTX является механизм наследования. Секция может быть объявлена как дочерняя по отношению к одной или нескольким родительским секциям с помощью синтаксиса `[child_section]:parent_section_a,parent_section_b` [11] [12]. В этом случае дочерняя секция автоматически получает все ключи и значения от своих родителей. Однако, если какой-либо ключ определен как в родителе, так и в ребенке, значение у ребенка всегда имеет приоритет и перезаписывает значение родителя [12]. Движок поддерживает множественное наследование, и в случае конфликтов между несколькими родителями, движок применяет стратегию разрешения, которая обычно основана на порядке перечисления родителей (слева направо) [11]. Этот механизм наследования является ядром логики слияния и должен быть точно воспроизведен в Python-реализации.

Еще одним важным элементом синтаксиса является директива `#include "filename.ltx"`, которая позволяет включать содержимое одного LTX-файла внутрь другого. Можно использовать и маски, например `w_*.ltx`, чтобы подключить все файлы, соответствующие шаблону [11]. Важно, что пути в директивах `#include` являются относительными к месту расположения файла, содержащего эту директиву [11]. Этот механизм создает древовидную структуру зависимостей между файлами, которую движок должен обходить для полного сбора конфигурации.

Наконец, следует обратить внимание на некоторые нюансы, которые критически важны для идентичности. Одним из них является чувствительность к регистру. В предоставленных источниках прямо указано, что движок во время парсинга преобразует все имена секций в нижний регистр [6]. Это означает, что секции `[WPN_AK74]` и `[wpn_ak74]` будут считаться одной и той же секцией и объединятся в одну. Любая Python-реализация, работающая с регистром букв, будет давать неверный результат. Другой важный аспект — это строгость парсинга. Например, в одном из случаев сбоя движка при загрузке профиля NPC было вызвано отсутствие заданного идентификатора, что привело к аварийному завершению работы [22]. Хотя это пример из XML-парсера, он иллюстрирует, что движок может иметь строгие правила и места, где он может аварийно завершиться. Аналогичные точки потенциального сбоя могут существовать и в LTX-парсере (например, некорректные пути в `#include`,

пустые секции). Python-реализация должна либо эмулировать эти же точки падения, либо, что более вероятно для достижения идентичности, обрабатывать их тем же самым образом, что и оригинал.

В таблице ниже обобщены ключевые различия между LTX и XML, имеющие значение для задачи симуляции конфигурации.

Характеристика	**LTX (`.ltx`)**	**XML (`.xml`)**
Основное назначение	Предметы, монстры, торговцы, настройки [7]	UI, локализация, диалоги, спавны [7]
Модель загрузки	Загружается и объединяется в `system.ltx` один раз при старте игры [7]	Читается по требованию при первом обращении [7]
Применение изменений	Требуется полный перезапуск игры [7]	Может потребоваться только перезагрузка соответствующего экрана [7]
Основной механизм слияния	Система DLTX для динамического слияния файлов [23]	Система DXML для слияния через Lua-скрипты [1]
Чувствительность к регистру	Имена секций приводятся к нижнему регистру [6]	Чувствительность к регистру зависит от конкретного файла и его обработчика
Цель симуляции	Создание финального объединенного файла (аналог `system.ltx`) [7]	Не применимо к задаче создания "гигаконфига"

Таким образом, для решения поставленной задачи необходимо сфокусироваться на симуляции процесса сборки LTX-конфигурации. Это включает в себя парсинг всех файлов из `gamedata/configs/`, разрешение директив `#include`, корректную обработку наследования и, что наиболее важно, полное воспроизведение логики слияния через систему DLTX. Любой упущенный нюанс в этих базовых механизмах сделает невозможным достижение бит-в-бит идентичного результата.

Глубокий анализ системы **DLTX:** Ядро динамической сборки конфигураций

Если базовый парсинг LTX можно сравнить с грамматикой языка, то система DLTX (Dynamic LTX) — это его семантика и синтаксический анализатор, определяющий смысл каждого предложения и как оно вписывается в общую картину. DLTX является ключевым и наиболее сложным компонентом, который необходимо воспроизвести для достижения идентичности конфигурации. Она

представляет собой надстройку над стандартным LTX, предназначенную для безопасного и гибкого слияния конфигурационных файлов от разных модов [15] [23]. Без точной симуляции DLTX любой полученный конфиг будет лишь частичным и неверным отражением того, что видит движок при запуске игры [8].

Центральная идея DLTX заключается в том, чтобы позволить модам не заменять базовые файлы, а применять к ним "патчи". Эти патчи представляют собой специальные файлы, которые движок автоматически находит и применяет к соответствующим базовым файлам в определенном порядке [6]. Механизм обнаружения патчей строго регламентирован: движок ищет в той же директории, где находится базовый файл, файлы с именем, соответствующим шаблону `mod_<имя_базового_файла>_<что-то>.ltx` [6] [8]. Например, для базового файла `wpn.ak74.ltx` движок будет искать патчи вроде `mod_wpn_ak74_balance.ltx` или `mod_wpn_ak74_fixes.ltx`. Чтобы избежать неоднозначности, существует правило: если в директории существуют `base.ltx` и `base_extra.ltx`, то патч `mod_base_patch.ltx` будет присвоен файлу `base_extra.ltx`, а не `base.ltx`, так как `base_extra` является более длинным совпадением [6].

Логика слияния DLTX основана на концепции "глубины" (`depth`). Каждому ключу или секции присваивается числовое значение глубины, которое определяет его приоритет. Высший приоритет имеет наименьшее (или наиболее отрицательное) значение глубины. 1. Базовые файлы имеют глубину `0` [6] [8]. 2. Модификационные файлы (патчи) сортируются по алфавиту и получают глубины `-200`, `-400`, `-600` и т.д. [6] [8]. Это означает, что патч, идущий по алфавиту позже, имеет более высокий приоритет и его изменения будут применены в последнюю очередь. 3. Файлы, включаемые директивой **`#include`**, получают положительную глубину (+1, +2, ...), что делает их содержимое самым низкоприоритетным и оно будет перезаписано любыми изменениями из базовых файлов или патчей [6].

Эта система глубин позволяет строго контролировать итоговое значение ключа. Алгоритм слияния для каждой секции выглядит следующим образом:

- Все ключи из всех файлов, влияющих на секцию, собираются в один список.
- Этот список сортируется по нескольким критериям: сначала по возрастанию глубины (ниже глубина — выше приоритет), затем по имени

ключа (алфавитный порядок), и, наконец, по индексу вставки (позже вставленные значения имеют преимущество) [6].
- На основе этого отсортированного списка формируется итоговая секция.

Особого внимания заслуживает синтаксис, который позволяет моддерам управлять этим процессом слияния с высокой степенью детализации. Эти операторы должны быть полностью реализованы в Python-реализации.

Оператор / Синтаксис	Описание	Пример
`[section]`	Создает новую секцию или патчит существующую, добавляя новые ключи и перезаписывая существующие [6].	`[wpn_ak74]`
`@[section]`	"Безопасный режим". Создает секцию, если она не существует (`override = false`), и патчит ее, если существует (`override = true`) [1] [2].	`@[wpn_ak74_new]`
`![section]`	Полностью заменяет секцию. Удаляет все наследование и ключи из родительских секций, заменяя их содержимым внутри `![]` [6] [8].	`![wpn_ak74]`
`!![section]`	Помечает секцию на удаление после всех остальных слияний. Ключи из такой секции не попадут в итоговый конфиг [6] [8].	`!![wpn_ak74_disabled]`
`!key`	Помечает ключ на удаление. Во время финальной обработки движок заменяет его специальным маркером `DLTX_DELETE` [6] [8].	`!scope_names`
`>key = item`	Добавляет элементы в конец списка, разделенного запятыми. Если ключ не является списком, он становится таким [2] [6].	`>use_modes = repair`
`<key = item`	Удаляет указанные элементы из списка, разделенного запятыми [2] [6].	`<use_modes = trade`
`[child]:!parent`	Внутри одного DLTX-файла дочерняя секция отказывается от наследования от конкретного родителя [6] [8].	`[wpn_temp]:!wpn_ak74`

Особенно важны операторы для манипулирования списками (> и <). Они применяются последовательно ко всем DLTX-файлам в порядке их применения (от базового к патчам), что позволяет создавать сложные цепочки изменений [6]. Например, один патч может добавить новый режим использования, а другой — удалить старый. Порядок применения этих патчей определяется их алфавитным именем, что делает систему предсказуемой, но требует от разработчиков патчей соблюдать соглашения об именовании для контроля над результатом.

Обработка наследования в контексте DLTX также является сложной задачей. Разрешение наследования происходит рекурсивно: движок просматривает всю цепочку родителей, дедушек и т.д., собирая все доступные ключи [6]. При наличии множественного наследования (секция наследует от нескольких родителей) конфликты решаются на основе порядка родителей в декларации и

глубины ключей. Если ключ определен в нескольких родителях, то берется тот, у которого наименьшая глубина (наибольший приоритет). Если ключ был удален в родительской секции (помечен `!key`), то дочерняя секция не может его восстановить обычным присвоением значения [8]. Это важное ограничение, которое должно быть учтено.

Одним из самых строгих правил, обеспечивающих стабильность системы, является проверка на циклические ссылки в наследовании. Если движок обнаруживает, что секция пытается наследовать от своей потомка (например, A наследует B, а B наследует A), он немедленно вызывает `Debug.fatal()` и аварийно завершает работу [6] [8]. Это предотвращает бесконечные циклы и некорректное разрешение конфигурации. Python-реализация должна иметь абсолютно аналогичный механизм строгой проверки циклов.

В таблице ниже представлена сводка по ключевым аспектам DLTX, которые необходимо проверить на идентичность.

Аспект **DLTX**	Описание	Необходимость для идентичности
Механизм обнаружения	Поиск файлов по шаблону `mod_<basename>_*.ltx` в директории базового файла [6] [8].	Критическая
Приоритет (глубина)	Базовые файлы (0), патчи (-200, -400...), `#include` (+1, +2...) [6] [8].	Критическая
Сортировка	Отсортировано по глубине, затем по имени ключа, затем по индексу вставки [6].	Критическая
Операторы слияния	Реализация `@`, `!`, `!!`, `>`, `<`, `!key` и синтаксиса наследования `:!`.	Критическая
Резолюция наследования	Рекурсивное разрешение, порядок родителей, запрет на восстановление удаленных ключей [6] [8].	Критическая
Проверка циклов	Обнаружение и аварийное завершение при циклических наследованиях [6] [8].	Критическая
Обработка ошибок	Логирование вместо падения при переопределении несуществующей секции [1].	Важная для совместимости

В конечном счете, для достижения цели исследования Python-реализация должна быть не просто парсером, а полноценным процессором **DLTX**. Она должна точно моделировать каждую стадию этого сложного процесса: от сбора и сортировки файлов до присвоения глубин, применения операторов слияния и, наконец, финальной резолюции наследования с обязательной проверкой на циклы. Любое упрощение или игнорирование одного из этих

шагов неизбежно приведет к появлению расхождений и, как следствие, к получению неидентичного конфигурационного файла.

Сопутствующие механизмы: Кэширование, обработка ошибок и специфические нюансы

Для достижения бит-в-бит идентичности финального конфигурационного файла недостаточно воспроизвести только основную логику слияния. Необходимо учесть и ряд сопутствующих механизмов, которые, хотя и не изменяют само содержимое конфига, влияют на процесс его создания и могут стать источником расхождений, если их не эмулировать. К таким механизмам относятся кэширование, строгая обработка ошибок и множество мелких, но важных нюансов в парсинге и слиянии данных.

Одним из ключевых аспектов производительности C++ движка X-Ray является кэширование. После первого запуска игры или сборки конфигурации, движок не заново парсит и не объединяет десятки или сотни LTX-файлов. Вместо этого он сохраняет готовый, полностью резолвленный результат для каждого файла в специальном кэше [6] [8]. При последующих запусках, если базовые LTX-файлы не были изменены, движок сразу загружает данные из этого кэша, что значительно ускоряет загрузку игры [8]. Этот кэш является внутренней оптимизацией движка, направленной на быстрый доступ к конфигурации во время игры. Для нашей задачи, где мы хотим симулировать именно первый процесс сборки конфигурации, сам механизм кэширования второстепенен. Однако важно понимать, что кэш хранит уже объединенную и резолвленную версию конфига. Если Python-скрипт правильно собирает конфиг с самого начала, его итоговый результат будет идентичен тому, что было бы помещено в кэш. Следовательно, отсутствие в Python-реализации собственного механизма кэширования само по себе не является недостатком для решения поставленной задачи. Однако, если бы нам нужно было симулировать второй запуск, то доступ к этому кэшу стал бы обязательным.

Гораздо более важным является механизм обработки ошибок и граничных случаев. Оригинальный движок, будучи C++ приложением, может аварийно завершаться при столкновении с некорректными данными. Анализ предоставленных материалов позволяет выявить несколько таких

потенциальных точек сбоя и специфических правил поведения, которые должны быть воспроизведены. 1. Обработка невалидных секций: В одном из отчетов о сбоях движка в X-Ray 1.6 была зафиксирована ситуация, когда при загрузке файла `user.ltx` возникала ошибка `id for item don't set` в номере 118 в `npc_profile` [22]. Это указывает на то, что движок имеет строгие требования к структуре некоторых файлов и может падать, если эти требования не выполняются. Аналогичные точки потенциального сбоя могут существовать и в LTX-парсере, например, при обработке некорректных путей в директивах `#include`, пустых секций или секций с некорректной структурой наследования. 2. Переопределение несуществующей секции: Как уже упоминалось, в более ранних версиях движка попытка переопределить несуществующую секцию могла приводить к краху. В `xray-monolith` эта логика была изменена: теперь такая операция лишь выводит сообщение в лог, но не вызывает аварийное завершение [1]. Это важное изменение в поведении, которое демонстрирует эволюцию движка и должно быть точно отражено в симуляторе. Python-реализация должна имитировать именно это "мягкое" поведение, а не быть более строгой. 3. Чувствительность к регистру: Это один из самых критичных нюансов. Источники однозначно указывают, что движок во время парсинга приводит все имена секций к нижнему регистру [6]. Это означает, что `[SECTION]`, `[Section]` и `[section]` — это одна и та же секция. Любая Python-реализация, которая различает регистр, будет работать некорректно и давать неверные результаты слияния. Это расхождение невозможно преувеличить и является одним из главных факторов, способных сделать результат неидентичным.

Другие специфические нюансы, которые следует учесть:

- Порядок обработки вложенных `#include`: Когда файл A включает файл B, а файл B включает файл C, движок должен обрабатывать эту цепочку в правильном порядке. Важно понимать, как именно он управляет стеком обработки включений, чтобы не потерять какие-либо файлы или обработать их в неверном порядке.
- Обработка дубликатов секций: Хотя основной механизм слияния через DLTX предотвращает многие конфликты, в процессе обработки директив `#include` могут возникнуть ситуации, когда одна и та же секция определена в нескольких файлах, включенных в один и тот же контекст. Оригинальный движок должен иметь четкое правило для разрешения таких дубликатов, которое, скорее всего, связано с порядком включения и глубиной.

- Специфический синтаксис **ACDC:** В одном из C# парсеров для LTX упоминается обработка специфического синтаксиса `<[18]. Хотя это не является частью стандарта X-Ray, наличие такого синтаксиса в экосистеме Anomaly может быть важно. Необходимо выяснить, поддерживает ли C++ движок X-Ray Monolith подобные конструкции и как он с ними работает.
- Порядок слияния внутри секции: Помимо глубины, движок использует другие критерии для окончательной сортировки ключей внутри секции: сначала по имени ключа (алфавитный порядок), а затем по индексу вставки (ключи, добавленные позже, имеют преимущество) [6]. Это означает, что если два ключа имеют одинаковую глубину, их порядок в итоговом файле будет определяться их именами.

В таблице ниже собраны эти сопутствующие механизмы и их влияние на задачу верификации.

Механизм / Нюанс	Описание	Влияние на идентичность
Кэширование	Хранение уже объединенной конфигурации для ускорения загрузки [6] [8].	Второстепенное. Основная задача — симулировать первый проход сборки.
Обработка ошибок	Движок может аварийно завершаться при некорректных данных (например, отсутствие ID) [22].	Критическое. Python-реализация должна эмулировать те же точки падения.
"Мягкое" поведение	Переопределение несуществующей секции теперь лишь пишет в лог, а не падает [1].	Критическое. Необходимо воспроизвести именно это поведение.
Приведение к нижнему регистру	Имена секций приводятся к нижнему регистру при парсинге [6].	Критическое. Расхождение здесь гарантированно приведет к неидентичному конфигу.
Порядок сортировки ключей	Сортировка по имени ключа, а затем по индексу вставки [6].	Важное. Может влиять на порядок ключей в итоговом файле.
Обработка **#include**	Правильная обработка вложенных включений и разрешение дубликатов [11].	Критическое. Ошибки здесь приведут к потере или неверной обработке данных.

Таким образом, успешная верификация требует не только воспроизведения основной логики слияния, но и внимания к этим деталям. Python-реализация должна быть настолько "грязной" и "неидеальной", насколько это возможно, чтобы точно имитировать поведение C++ движка. Это означает, что она должна не только правильно обрабатывать корректные данные, но и корректно реагировать на некорректные, падая в тех же местах, где падает оригинал, или обрабатывая их так же мягко. Только такой подход позволит достичь истинной функциональной эквивалентности.

Анализ Python-реализации: `gitxray.py` и `Resolver2.py` как инструменты симуляции

На основе предоставленных материалов можно составить лишь частичное представление о структуре и возможностях Python-реализации, однако этого достаточно для проведения качественного аналитического сравнения и выявления потенциальных пробелов. Центральное место в этой реализации занимают два файла: `gitxray.py` и `Resolver2.py` [17]. Понимание их роли и потенциального содержания является ключевым для оценки готовности проекта к решению поставленной задачи.

Структура проекта, описанная в документации к `gitxray.py`, указывает на его роль как основного скрипта-оркестратора [17]. Он, вероятно, отвечает за инициализацию окружения, создание контекстных объектов (`gx_context`) и вызов необходимых модулей для выполнения конкретных задач, таких как разбор конфигураций. Это типичная архитектура для сложного инструментария, где `gitxray.py` является точкой входа для пользователя, а `Resolver2.py` содержит основную бизнес-логику. Таким образом, фокус анализа должен быть смещен на `Resolver2.py` как на сердце системы симуляции конфигураций.

Сравним эту структуру с архитектурой C++ оригинала. В движке `xray-monolith` центральное место занимает класс `CInifile`, определенный в `src/xrCore/xr_ini.h` и реализованный в `src/xrCore/Xr_ini.cpp` [20]. Этот класс является не просто парсером, а полноценной системой управления конфигурацией. Его методы, такие как `LTXLoad()`, `EvaluateSection()`, `MergeSections()` и `SortAndFilterSection()`, являются прямыми аналогами тех самых операций, которые нужно симулировать [20]. `CInifile` отвечает за парсинг, кэширование, обработку `#include` и, что самое главное, за всю сложную логику слияния через DLTX. Он использует внутренние структуры данных, такие как `Sect` (секции) и `Item` (ключ-значение), каждое из которых может содержать метаданные, включая `depth` (глубина), `filename` (источник) и `insertionIndex` (порядок вставки) [20].

Исходя из этого, можно сделать вывод, что `Resolver2.py` должен быть попыткой создать Python-аналог этого сложного C++ класса. Однако, здесь и кроется основная проблема. Предоставленные материалы не содержат кода `Resolver2.py`, поэтому мы можем судить о его возможностях только косвенно,

сравнивая его с C++ оригиналом и анализируя описание других Python-инструментов для работы с LTX.

Существуют и другие Python-библиотеки для работы с LTX, такие как `pystalker` и `LtxParser` [18] [19]. `pystalker` представляет собой библиотеку и CLI-инструмент для обследования LTX-файлов, позволяя делать запросы к данным так, как если бы они были загружены в игру [19]. `LtxParser` — это C# библиотека, которая также может служить ориентиром [18]. Анализ этих инструментов показывает, что большинство из них сфокусированы на базовом парсинге LTX: они умеют читать файлы, разбирать секции и ключи, обрабатывать наследование на самом базовом уровне (например, просто копируя ключи от родителей). Однако, ни один из них не упоминает реализацию сложной логики DLTX с ее операторами (`>`, `<`, `!`, `!!`), системой глубин и правилами слияния.

Это наблюдение крайне важно. Оно позволяет предположить, что `Resolver2.py`, скорее всего, либо: 1. Является упрощенным парсером, который не реализует всю сложную логику DLTX, а лишь базовый парсинг LTX. 2. Имеет неполную реализацию **DLTX**, где отсутствуют некоторые операторы или правила слияния. 3. Имеет ошибки в реализации, например, неправильно обрабатывает приоритеты, глубины или порядок сортировки.

Отсутствие в предоставленных материалах упоминания DLTX-специфичной логики в контексте Python-реализации является самым очевидным и критическим пробелом. Для решения поставленной задачи требуется не просто парсер, а полноценный симулятор движка конфигураций **X-Ray Monolith**. Это означает, что `Resolver2.py` должен быть гораздо более сложным, чем простой парсер. Он должен в точности воспроизводить следующий алгоритм:

- Шаг **1:** Сбор и классификация файлов. Скрипт должен собрать все `.ltx` файлы в директории `gamedata/configs/` и классифицировать их как базовые или модификационные (`mod_*.ltx`), используя правило `mod_<имя_базового_файла>_<...>.ltx` [6].
- Шаг **2:** Присвоение глубин. Базовые файлы получают глубину 0. Модификационные файлы сортируются по алфавиту, и каждому присваивается глубина -200*N, где N — порядковый номер [6].
- Шаг **3:** Парсинг и хранение. Каждый файл должен быть разобран на секции и ключи. Вместо простого словаря, структура данных должна быть сложнее. Например, для каждой секции в памяти должен храниться

словарь, где ключами являются глубины, а значениями — временные объекты секций, содержащие ключи, добавленные на данном уровне слияния [20].

- Шаг **4:** Применение правил слияния. Это самый сложный этап. Скрипт должен последовательно обрабатывать все операторы DLTX (`@`, `!`, `!!`, `>`, `<`, `!ключ`). Это не просто простое присвоение; это сложный процесс манипулирования структурами данных, имитирующий, как движок обрабатывает каждый патч.
- Шаг **5:** Финальная резолюция. После того как все патчи применены, для каждой секции должен быть выполнен финальный проход по графу наследования. На этом этапе происходит рекурсивное разрешение родительских секций, проверка на циклы (с вызовом `fatal` при их обнаружении) и окончательное формирование итоговой секции [6] [8].
- Шаг **6:** Корректный вывод. Итоговый конфигурационный файл должен быть сгенерирован с учетом всех правил, включая приведение имен секций к нижнему регистру и сортировку ключей по имени [6].

В таблице ниже приведено сравнение компонентов C++ `CInifile` и того, что должно быть реализовано в `Resolver2.py`.

Компонент C++ (`CInifile`)	Функциональное соответствие в Python (`Resolver2.py`)	Критичность
Парсинг LTX	Чтение и разбор синтаксиса (секции, ключи, комментарии) [11].	Критическая
Обработка `#include`	Рекурсивное чтение и слияние включаемых файлов.	Критическая
Система глубин (`depth`)	Присвоение и сравнение глубин для определения приоритета слияния.	Критическая
Система DLTX	Полная реализация всех операторов (@, !, !!, >, <, !key) и правил слияния.	Критическая
Резолюция наследования	Рекурсивное разрешение родительских секций с проверкой на циклы.	Критическая
Кэширование	Сохранение и загрузка готового конфига.	Второстепенная (для первой сборки)
Обработка ошибок	Эмуляция падений и логирования в соответствии с оригиналом.	Важная
Нижний регистр	Приведение имен секций к нижнему регистру.	Критическая

Анализ показывает, что для достижения цели `Resolver2.py` должен быть гораздо более сложным инструментом, чем может показаться на первый взгляд. Он должен быть не парсером, а полноценным мини-движком, симулирующим логику конфигурационной системы X-Ray. Без наличия в нем всех

перечисленных компонентов, особенно сложной логики DLTX, он не сможет произвести идентичный конфиг. Без доступа к самому коду `Resolver2.py` невозможно дать окончательный вердикт, но на основе имеющихся данных можно с уверенностью утверждать, что наиболее вероятной причиной неудачи является неполнота реализации DLTX.

Итоговый вердикт и рекомендации по достижению функциональной эквивалентности

Проведенный всесторонний сравнительный анализ оригинального C++ кода движка X-Ray Monolith и контекста его Python-реализации позволяет сформулировать четкий итоговый вердикт и предоставить конкретные рекомендации для достижения поставленной исследовательской цели. Задача заключается не просто в сравнении двух кодовых баз, а в подтверждении их функциональной эквивалентности с целью получения финального конфигурационного файла, бит-в-бит идентичного тому, что сгенерировал бы запущенный игровой клиент.

Итоговый вердикт:

На основании предоставленных данных, **Python-**реализация**,** представленная в виде **`Resolver2.py`**, с высокой вероятностью не является готовой к производству идентичного конфига. Основной и критический недостаток заключается в почти полном отсутствии в анализируемых материалах упоминания или описания реализации сложной системы DLTX (Dynamic LTX) [1] [2]. Задача воспроизведения поведения движка X-Ray Monolith для генерации конфигураций — это задача симуляции, а не простого парсинга. Симуляция должна включать в себя полное воспроизведение всех аспектов работы класса `CInifile` из C++ оригинала, в частности, всех функций слияния, таких как `LTXLoad()`, `EvaluateSection()`, `MergeSections()` и `SortAndFilterSection()` [20]. Простого парсера LTX, который обрабатывает только базовый синтаксис, недостаточно для достижения искомой идентичности.

Обоснование вердикта:

1. Сложность системы **DLTX:** DLTX — это не просто расширение синтаксиса, а полноценная система управления конфигурациями со своими правилами, приоритетами (глубиной), операторами (@, !, >, <, ! !) и механизмом разрешения конфликтов [6] [8]. Ее логика является ядром процесса сборки конфигурации в модифицированном движке Anomaly. Любой Python-инструмент, не реализующий эту логику в полной мере, неизбежно будет генерировать неверный результат.
2. Нюансы и граничные случаи: Достижение бит-в-бит идентичности требует эмуляции даже малейших "кривых" оригинального движка. К таким аспектам относятся:

- Приведение имен секций к нижнему регистру: Это фундаментальное свойство, которое меняет саму природу слияния секций. Игнорирование этого факта приведет к абсолютно разным результатам [6].
- Обработка ошибок и **edge-cases:** Оригинальный движок имеет строгие правила, например, проверку на циклические ссылки в наследовании, которая приводит к аварийному завершению [6] [8]. Python-реализация должна иметь аналогичный механизм. Также важно воспроизвести измененное поведение, при котором попытка переопределить несуществующую секцию не падает, а лишь логируется [1].
- Порядок сортировки и обработки: Итоговый порядок ключей в секции определяется не только глубиной, но и алфавитным порядком имен ключей, а также индексом вставки [6]. Точный повтор этого порядка важен для идентичности.

1. Неполнота **Python-**инструментов: Анализ других существующих Python-инструментов для работы с LTX, таких как `pystalker` [19], показывает, что они сфокусированы на базовом парсинге и обследовании данных, а не на сложной логике слияния. Это подтверждает, что `Resolver2.py` с высокой вероятностью также не содержит необходимой функциональности.

Рекомендации по достижению функциональной эквивалентности:

Для того чтобы Python-реализация стала идентичной C++ оригиналу, необходимо провести ее глубокую доработку, следуя алгоритму, который имитирует работу `CInifile` и системы DLTX. Ниже представлен пошаговый план действий.

Шаг **1:** Реализация структур данных, аналогичных **C++** Вместо простых словарей и списков необходимо создать более сложные структуры данных, которые будут хранить не только ключи и значения, но и всю необходимую метаинформацию, как в C++-оригинале [20]:

- Класс `**Item**` (ключ): Должен хранить значение, имя ключа и, что самое главное, атрибут `depth` (глубина).
- Класс `**Section**` (секция): Должен хранить имя секции (в нормализованном виде, например, в нижнем регистре) и словарь или список `Items`. Для хранения слияний лучше использовать словарь, где ключ — это `depth`, а значение — это временная секция, содержащая `Items` с этой глубиной.

Шаг **2:** Реализация механизма слияния по **DLTX** Это самый ответственный этап. Необходимо реализовать алгоритм, который точно повторяет логику движка: 1. Сборка и сортировка файлов: Найти все `.ltx` файлы. Определить базовые файлы и сгенерировать список модификационных файлов (`mod_*.ltx`). Отсортировать этот список по алфавиту. 2. Присвоение глубин: Пройтись по списку файлов и присвоить им глубины: базовым — 0, модификационным — -200, -400, и т.д. [6] [8]. 3. Парсинг и накопление данных: Для каждого файла, прочитать его содержимое. При парсинге секции, приводить ее имя к нижнему регистру [6]. Для каждой найденной секции и ключа создать объекты `Section` и `Item`, присвоив им глубину. Добавить их в глобальную структуру данных (например, словарь секций, где каждый ключ — это имя секции в нижнем регистре, а значение — словарь глубин -> секций). 4. Применение правил слияния (проход по глубинам): Для каждой секции в итоговой структуре данных нужно выполнить проход по глубинам в порядке их возрастания (от 0 к -N). На каждом шаге применять правила DLTX:

- Если встречен оператор `@`, `!` или простая секция, данные сливаются в общую секцию.
- Если встречен оператор `!key`, ключ помечается на удаление.
- Если встречен оператор `>key` или `
- Если встречен оператор `!!`, секция помечается на удаление.
- Все эти действия должны модифицировать общую структуру данных, отражающую итоговое состояние секции.

Шаг **3:** Финальная резолюция наследования После того как все слияния выполнены и все операторы применены, необходимо разрешить наследование для каждой секции: 1. Рекурсивный обход: Для каждой секции рекурсивно обойти всю ее цепочку родителей, собирая их ключи и значения. 2. Проверка на

циклы: Во время обхода необходимо отслеживать посещенные секции. Если при обходе встречается уже посещенная секция, это означает наличие цикла. В этом случае программа должна аварийно завершиться с сообщением об ошибке, точно так же, как это делает C++ движок [6] [8]. 3. Разрешение конфликтов: При слиянии ключей от родителей, если ключ с тем же именем уже существует, новое значение должно перезаписать старое. Порядок обхода родителей важен и должен быть идентичен оригиналу [11]. 4. Удаление помеченных ключей: Перед финальным выводом для каждой секции необходимо удалить все ключи, которые были помечены на удаление оператором `!key` [8].

Шаг 4: Генерация итогового файла и тестирование 1. Сортировка и вывод: Для каждой итоговой секции отсортировать ключи по их именам (алфавитный порядок) и сгенерировать строку файла [6]. 2. Тестирование: Полученный Python-скрипт необходимо протестировать на большом количестве различных конфигурационных файлов, включая как базовые, так и файлы с патчами. Для верификации идентичности можно использовать инструменты вроде `LTXDiff`, которые предназначены для сравнения стандартных модов с DLTX-версиями [1]. Сравнение вывода Python-скрипта с результатом запуска игры (если это возможно в рамках эксперимента) станет окончательным доказательством успеха.

В заключение, задача достижима, но требует серьезного подхода. Она не является простым переносом кода, а представляет собой задачу по детальному переведению сложной бизнес-логики C++ движка на Python. Главный риск заключается в недооценке сложности системы DLTX. Успешная реализация должна быть не просто парсером, а полноценным симулятором конфигурационной системы X-Ray Monolith, который воспроизводит каждый шаг, от парсинга до финальной резолюции наследования, с учетом всех нюансов и специфических правил оригинального движка.

Справка

1. themrdemonized/xray-monolith https://github.com/themrdemonized/xray-monolith

2. GitHub - Hozar2002/xray-monolith-themrdemonized: https: ... https://github.com/Hozar2002/xray-monolith-themrdemonized
3. openxray/xray-16: Improved version of the X https://gitflic.ru/project/openxray/xray-16
4. GitHub - xrLil-Batya/xray-monolith https://github.com/xrLil-Batya/xray-monolith/
5. GitHub - Ishmaeel/xray-monolith https://github.com/Ishmaeel/xray-monolith
6. DLTX (LTX Patching) - S.T.A.L.K.E.R. Anomaly Modding Guide https://robe127.github.io/stalker-anomaly-modding-guide/config-formats/dltx/
7. Config Formats - S.T.A.L.K.E.R. Anomaly Modding Guide https://robe127.github.io/stalker-anomaly-modding-guide/config-formats/
8. stalker-anomaly-modding-guide/docs/config-formats/dltx.md at ... https://github.com/robE127/stalker-anomaly-modding-guide/blob/main/docs/config-formats/dltx.md
9. GitHub - robE127/stalker-anomaly-modding-guide: An attempt to ... https://github.com/robE127/stalker-anomaly-modding-guide
10. themrdemonized/xray-monolith | DeepWiki https://deepwiki.com/themrdemonized/xray-monolith
11. LTX Format - S.T.A.L.K.E.R. Anomaly Modding Guide https://robe127.github.io/stalker-anomaly-modding-guide/config-formats/ltx/
12. Ltx reading through script - Mod Wiki - sdk.stalker-game.com https://sdk.stalker-game.com/en/index.php?title=Ltx_reading_through_script
13. xrLil-Batya/xray-monolith https://github.com/xrLil-Batya/xray-monolith
14. CInifile and DLTX System | themrdemonized/xray-monolith ... https://deepwiki.com/themrdemonized/xray-monolith/2.1-cinifile-and-configuration-management
15. DLTX Processing Pipeline | themrdemonized/xray-monolith ... https://deepwiki.com/themrdemonized/xray-monolith/2.2-dltx-processing-pipeline
16. Game crashing.. Please help Thread - STALKER Anomaly https://www.moddb.com/mods/stalker-anomaly/forum/thread/game-crashing-please-help
17. Code structure https://www.gitxray.com/code_structure/
18. JKAnderson/LtxParser: C# library for loading .ltx trees from ... https://github.com/JKAnderson/LtxParser
19. GitHub - grant-h/pystalker: STALKER / OpenXRay file parsing ... https://github.com/grant-h/pystalker
20. Configuration and Modding Systems | themrdemonized/xray ... https://deepwiki.com/themrdemonized/xray-monolith/2-configuration-and-modding-systems

21. DLTX Processing Pipeline | themrdemonized/xray-monolith ... https://deepwiki.com/themrdemonized/xray-monolith/2.2-dltx-mod-loading-and-priority
22. OpenXRay/xray-16 - Fatal Error when starting application https://github.com/OpenXRay/xray-16/issues/509
23. DLTX System and Mod Loading | themrdemonized/xray-monolith ... https://deepwiki.com/themrdemonized/xray-monolith/2.2-dltx-system-and-mod-loading
24. Mord3rca/gamma-launcher https://github.com/Mord3rca/gamma-launcher
25. gamma-launcher/tests/test_userltx.py at master · Mord3rca ... https://github.com/Mord3rca/gamma-launcher/blob/master/tests/test_userltx.py
26. PavelBlend/blender-xray: STALKER (aka xray-engine ... https://github.com/PavelBlend/blender-xray
27. GitHub - Rammaken/xrToolkit: xrToolkit it's a X-Ray Engine ... https://github.com/Rammaken/xrToolkit
28. GitHub - ChristopherCapito/stalker-anomaly-devtools: A ... https://github.com/ChristopherCapito/stalker-anomaly-devtools
29. Stalker 2 Configuration FIle Merge Tool https://www.nexusmods.com/stalker2heartofchornobyl/mods/326?tab=posts